Análise de rede neural em NumPy

Arquitetura, classificação (não é rede bayesiana) e observações gerais

Resumo

- **Camadas de neurônios: 3** (entrada com 1000, oculta com 100, saída com 10). Isso equivale a **2 camadas de pesos** (`w1` e `w2`) e **1 camada oculta**.
- **Tipo:** perceptron multicamadas (MLP) feedforward totalmente conectado, com ativação sigmoide na oculta e saída linear (regressão).
- **Rede bayesiana: não é.** É uma rede neural determinística com pesos pontuais, sem distribuição sobre parâmetros nem grafo de dependências probabilísticas.
- **Parâmetros:** 1000x100 + 100x10 = **101.000** (sem vieses/bias).
- **Treino:** gradiente descendente em lote completo (64 amostras), 2000 épocas, taxa 1e-4, perda = soma dos erros quadráticos.

Código analisado

```
import numpy as np
from numpy.random import randn

n, D_in, H, D_out = 64, 1000, 100, 10
x, y = randn(n, D_in), randn(n, D_out)
w1, w2 = randn(D_in, H), randn(H, D_out)

for t in range(2000):
  h = 1 / (1 + np.exp(-x.dot(w1)))
  y_pred = h.dot(w2)
  loss = np.square(y_pred - y).sum()
  print(t, loss)
  grad_y_pred = 2.0 * (y_pred - y)
  grad_w2 = h.T.dot(grad_y_pred)
  grad_h = grad_y_pred.dot(w2.T)
  grad_w1 = x.T.dot(grad_h * h * (1-h))
  w1 -= 1e-4 * grad_w1
  w2 -= 1e-4 * grad_w2
```

1. Estrutura da rede

Camada	Dimensão	Operação no código	Ativação
Entrada (x)	64 x 1000	`randn(n, D_in)`	-
Oculta (h)	64 x 100	`1/(1+exp(-x.dot(w1)))`	Sigmoide
Saída (y_pred)	64 x 10	`h.dot(w2)`	Linear (identidade)

Em termos de contagem, a literatura costuma dizer “redes de 2 camadas” (duas camadas de pesos, sem contar a entrada), que é como as notas do curso CS231n de Stanford (Fei-Fei Li, Andrej Karpathy, Justin Johnson) descrevem esse tipo de rede. Contando as camadas de neurônios, incluindo a entrada, são 3. Vale citar a convenção usada no seu trabalho.

2. Por que não é uma rede bayesiana

- Uma **rede bayesiana** é um grafo acíclico dirigido em que cada nó é uma variável aleatória com tabela/distribuição condicional dado seus pais. Aqui os nós são unidades de cálculo

determinísticas, não variáveis aleatórias.

- Uma **rede neural bayesiana** manteria uma distribuição sobre os pesos, p(w|dados), e produziria incerteza preditiva. Aqui `w1` e `w2` são apenas matrizes de números, atualizadas por gradiente.
- A perda quadrática equivale à **máxima verossimilhança (MLE)** sob ruído gaussiano. Com regularização L2 viraria **MAP** (prior gaussiano nos pesos), mas ainda seria estimativa pontual, não inferência bayesiana completa.
- A única aleatoriedade é a inicialização e os dados sintéticos (`randn`).

Classificação correta: rede neural feedforward (MLP) de uma camada oculta, treinada por backpropagation, aprendizado supervisionado para regressão.

3. Forward e backward pass

Forward: h = sigma(x W_1), y_{pred} = h W_2, L = soma de $(y_{pred} - y)^2$.

Backward (regra da cadeia):

Linha	Matemática
grad_y_pred	$dL/dy_{pred} = 2(y_{pred} - y)$
grad_w2	h^T (dL/dy_{pred})
grad_h	(dL/dy_{pred}) W_2^T
grad_w1	x^T [(dL/dh) * h * (1-h)] (derivada da sigmoide: sigma' = sigma(1-sigma))

Os gradientes estão **corretos** para essa arquitetura. É a mesma implementação “na mão” que as notas do CS231n usam para ilustrar backprop sem frameworks.

4. Resultado de uma execução (seed 0)

Época	Perda
0	24.636
10	8.926
100	2.383
500	211
1000	41
1999	4,3

A perda cai de ~24,6 mil para ~4, ou seja, a rede quase zera o erro. Mas os dados são **ruído aleatório**: x e y são independentes. Logo, isso é **memorização pura** (overfitting extremo), não aprendizado de padrão. Com 101.000 parâmetros e só 64 amostras, sobra capacidade para decorar tudo. O código é um exemplo didático de que o algoritmo de otimização funciona, não um modelo útil.

5. Pontos de atenção

- **Sem vieses (bias):** reduz a flexibilidade; o normal é `x.dot(w1) + b1` e `h.dot(w2) + b2`.
- **Inicialização ruim:** `randn` sem escala (sem Xavier/He) com 1000 entradas gera pré-ativações com desvio ~sqrt(1000) = 32. Medi que ~88% das ativações da sigmoide começam saturadas (<0,01 ou >0,99), o que mata o gradiente (h(1-h) ~ 0) e explica a taxa pequena (1e-4). Correção:

dividir por sqrt(D_in).

- **Sigmoide na oculta:** sofre com gradiente evanescente; ReLU é a escolha moderna.
- **Perda por soma, não média:** o gradiente escala com n; trocar por `.mean()` exige reajustar a taxa de aprendizado.
- **Lote completo:** sem mini-batches/SGD; ok para 64 amostras, não escala.
- **Sem validação, regularização ou dropout:** não há como medir generalização.
- **Saída linear com MSE:** adequado para regressão; para classificação usaria softmax + entropia cruzada.
- **Sem seed:** resultados não reproduzíveis; use `np.random.seed(0)`.
- **Desempenho:** impressão a cada época (2000 linhas) é desnecessária; imprima a cada 100.

6. Observação sobre a referência

Não conheço um livro de Machine Learning escrito por Fei-Fei Li. Esse trecho de código vem das **notas do curso CS231n (Stanford)**, ministrado por ela junto com Andrej Karpathy e Justin Johnson. Se o seu professor indicou um livro específico, confirme o título antes de citar. Para a parte bayesiana (redes bayesianas, MLE/MAP), as referências clássicas são Russell & Norvig, *Artificial Intelligence: A Modern Approach*, e Bishop, *Pattern Recognition and Machine Learning*.